Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées – Al Hoceima Filière : ID1 – S2

Année universitaire : 2025 - 2026

Compte Rendu

Manipulation des notions de POO :

Objets, classes, constructeurs, encapsulation, this, final, static, ...

TP N° 3

Objectif :

L'objectif de ce TP est de comprendre les notions de bases de la programmation orienté objet.

Réalisé par : Hajar LEBYED

Pr : m.cherradi

Exercic 1 :

J'ai créé une classe **Point** qui représente un point dans un repère cartésien 2D.

J'ai utilisé **private** pour protéger la valeur de x

```
package Ex1;

class Point{
    private double x;
    private double y;

```

J'ai créé un constructeur **par défaut** pour initialiser un point à l'origine.

```
    public Point(){
        this.x=0;
        this.y=0;
    }
```

J'ai créé un constructeur **paramétré** pour permettre à l'utilisateur de choisir les coordonnées du point ;

```
    public Point(double x,double y){
        this.x=x;
        this.y=y;
    }
```

Les méthodes :

afficher() : cette méthode permet d'**afficher les coordonnées** du point sous la forme **(x, y)**.

```
    public void afficher(){
        System.out.println("("+x+","+y+")");
    }
```

deplacer(double dx, double dy**)** : cette méthode permet de **modifier la position** du point en lui ajoutant un déplacement.

```
    public void deplacer(double dx, double dy) {
        this.x=this.x+dx;
        this.y=this.y+dy;
    }
```

distanceAorigine() : J'ai créé cette méthode pour calculer la distance entre le point et l'origine (0,0).

J'ai utilisé Math.sqrt() pour calculer la racine

```
    public double distanceAorigine(){
        return Math.sqrt(x*x+y*y);
    }
```

distanceXY() : cette méthode pour **calculer la différence** entre la coordonnée x et la coordonnée y.

J'ai utilisé Math.abs() pour obtenir une **valeur toujours positive**

```
    public double distanceXY(){
        return Math.abs(x-y);
    }
```

Exercic 2 :

J'ai crée une classe **Rectangle** représentant un rectangle, elle permet de calculer la surface et le périmètre.

```
package Ex1;

class Rectangle{
    private double largeur;
    private double hauteur;
```

Constructeur paramétré : initialise le rectangle avec une largeur et une hauteur donnée.

```
    public Rectangle(double largeur, double hauteur){
        this.largeur=largeur;
        this.hauteur=hauteur;
    }
```

Les méthodes :

Surface() : Calcule et retourne la surface du rectangle.

```
    public double Surface(){
        return largeur*hauteur;
    }
```

Perimetre() : Calcule et retourne le périmètre du rectangle.

```
    public double Perimetre(){
        return largeur*2+hauteur*2;
    }
```

Exercic 3 :

J'ai crée une classe **Etudiant** représentant un étudiant avec son CNE et sa note.

```
class Etudiant {
    private String cne;
    private float note;
```

J'ai crée un constructeur paramétré : initialise un étudiant avec son CNE et sa note.

```
    public Etudiant(String cne, float note) {
        this.cne = cne;
        this.note = note;
    }
```

Les méthodes :

toString() : Retourne une représentation textuelle de l'étudiant.

Affiche le CNE et la note sous forme lisible.

```
    public String toString() {
        return "CNE : " + cne + ", Note : " + note;
    }
```

estAdmis() : Vérifie si l'étudiant est admis.Elle retourne **true** si admis, **false** sinon

```
    public boolean estAdmis() {
        return note >= 12;
    }
```

getCne() : retourne le CNE de l'étudiant.

```
    public String getCne() {
        return cne;
    }
}
```

Exercic 4:

J'ai créé une classe **NombreComplexe** représentant un nombre complexe de la forme : **Re + i.Im** Elle permet d'effectuer des opérations mathématiques sur les nombres complexes.

```
class NombreComplexe{
    private float Re;
    private float Im;
```

J'ai créé un constructeur paramétré : initialise un nombre complexe avec sa partie réelle et imaginaire

```
    NombreComplexe (float Re,float Im){
        this.Re=Re;
        this.Im=Im;
    }
```

Les méthodes :

addition() : Additionne deux nombres complexes.

Formule : (a + ib) + (c + id) = (a+c) + i(b+d)

```
    public NombreComplexe addition(NombreComplexe z,NombreComplexe x){
        return  new NombreComplexe((z.Re + x.Re) , (z.Im+x.Im));
    }
```

Multiplication() : Multiplie deux nombres complexes.

Formule : (a + ib) × (c + id) = (ac - bd) + i(ad + bc)

```
    public NombreComplexe Multiplication(NombreComplexe z,NombreComplexe x){

        return  new NombreComplexe((z.Re*x.Re)-(z.Im*x.Im),(z.Re*x.Im)+(z.Im*x.Re));
    }
```

Afficher(): Affiche le nombre complexe sous la forme : Re + iIm

```
    public void Afficher() {
        System.out.println(Re+"+ i"+Im);
    }
```

Exercic 5 :

J'ai créé une classe **CompteBancaire** représentant un compte bancaire avec un numéro et un solde.

```
class CompteBancaire {
    private int numero;
    private double solde;
```

J'ai créé un constructeur paramétré : initialise un compte bancaire avec un numéro et un solde.

```
    CompteBancaire(int numero,double solde){
        this.numero=numero;
        this.solde=solde;
    }
```

Les méthodes :

Deposer() : Dépose un montant sur le compte.

Le montant doit être strictement positif pour être accepté.

```
    public void deposer(double montant){
        if(montant>0) {
            solde+=montant;
        }else
            System.out.println("Montant invalide");
    }
```

retirer( ) : Retire un montant du compte.

Le retrait est refusé si le montant dépasse le solde disponible.

```
public void retirer(double montant) {
    if(montant>solde) {
        System.out.println("impossible");
    }else
        solde-=montant;
    System.out.println("effuctue");
    }
```

transferer( ) : Transfère un montant depuis ce compte vers un compte destination.

Le transfert est refusé si le solde est insuffisant.

```
public void transferer(CompteBancaire destination,double montant){
    if(montant>solde) {
        System.out.println("solde inuffissant");
    }else {
        this.retirer(montant);
        destination.deposer(montant);
        System.out.println("le transfert de " +montant+" de "+this.numero+" vers "+ destination.numero);
    }
}
```

afficherSolde() : le solde actuel du compte.

```
public void afficherSolde(){
    System.out.println("Compte"+numero+ " votre solde : "+ solde);
}
```

Exercic 6 :

J'ai créé une classe **Voiture** représentant une voiture avec une marque et une vitesse. La vitesse maximale autorisée est de 240 km/h.

```
class Voiture{
    private  String marque;
    private int vitesse;
```

J'ai créé un constructeur paramétré : initialise une voiture avec sa marque et sa vitesse initiale.

```
    Voiture(String marque,int vitesse){
        this.marque=marque;
        this.vitesse=vitesse;
    }
```

Les méthodes :

accelerer() : Accélère la voiture en ajoutant une valeur à la vitesse actuelle. La vitesse maximale est limitée à 240 km/h.

```
    public void accelerer(int valeur){
        if(vitesse+valeur<=240){
            vitesse+=valeur;
            System.out.println("la voiture " +marque+" accelere : "+ vitesse+"km/h");
        }else {
            vitesse=240;
            System.out.println("la voiture "+marque+"a atteint la vitesse maximale : 240 km/h");
        }
    }
```

freiner() : Freine la voiture en réduisant la vitesse actuelle. La vitesse ne peut pas descendre en dessous de 0 km/h.

```
    public void freiner(int valeur){
        if(vitesse-valeur>=0) {
            vitesse-=valeur;
            System.out.println("la vitesse actuelle est : "+vitesse);
        }else
            vitesse=0;
            System.out.println("la voiture s'arrete :"+vitesse+"km/h");
    }
```

afficher() : les informations de la voiture : marque et vitesse actuelle.

```
    public void afficher() {
        System.out.println("la marque :"+this.marque+"| la vitesse :"+this.vitesse+"km/h");
    }
}
```

Exercic 7 :

J'ai créé une classe Produit représentant un produit en stock, elle permet de gérer l'ajout, la modification et la suppression de produits.

id, **nom**, **description**, **prix**, **quantite** sont des attributs d'instance propres à chaque produit.

```
class Produit{
    private int id;
    private String nom;
    private String description;
    private double prix;
    private int quantite;
```

nombreProduits est un attribut **statique**, partagé par tous les produits, compte le nombre total de produits créés.

```
    private static int nombreProduits = 0;
```

PRIX_MIN est un constante **finale** fixant le prix min.

```
private static final double PRIX_MIN = 0;
```

Constructeur paramétré : initialise un produit avec toutes ses informations et incrémente le compteur.

```
    public Produit (int id,String nom,String description,float prix,int quantite ){
        this.id=id;
        this.nom=nom;
        this.description=description;
        this.prix=prix;
        this.quantite=quantite;
        nombreProduits++;
    }
```

Les méthodes :

ajouterProd() : affiche un message confirmant l'ajout du produit.

```
   public void ajouterProd(){
       System.out.println("Produit ajouté : "+this.nom);
   }
```

modifierProd() : met à jour le prix (si valide) et la quantité du produit.

```
  public void modifierProd(double nouveauPrix,int nouvelleQuantite) {
      if(nouveauPrix>= PRIX_MIN){
          this.prix = nouveauPrix;
      }
      this.quantite = nouvelleQuantite;
      System.out.println("Produit modifié :"+this.nom);
  }
```

supprimerProd() : réinitialise les champs du produit et décrémente le compteur.

```
    public void supprimerProd(){
        this.nom ="";
        this.description ="";
        this.prix = 0;
        this.quantite = 0;
        nombreProduits--;
        System.out.println("Produit supprimé.");
    }
```

afficher() : affiche les informations principales du produit.

```
public void afficher(){
    System.out.println("ID: "+this.id+" | Nom : "+this.nom+" | Prix: "+this.prix+" DH | Quantité: "+this.quantite);
}
```

getNombreProduits() : méthode **statique** qui retourne le nombre total de produits.

```
   public static int getNombreProduits() {
       return nombreProduits;
   }
```

Exercic 7(BONUS) :

J'ai créé un programme de jeu où l'utilisateur doit deviner un nombre aléatoire entre 1 et 10.

n : nombre aléatoire généré entre 1 et 10.

sc : scanner pour lire la saisie de l'utilisateur.

nb : initialisé à -1 pour permettre l'entrée dans la boucle.

La boucle while : répète la saisie tant que le nombre saisi est différent du nombre à deviner.

Si **nb < n** → affiche "Plus grand !"

Si **nb > n** → affiche "Plus petit !"

Si **nb == n** → la boucle se termine et affiche "Bravo !"

```
public class Ex7Bonus {
    public static void main(String[] args){
    Random rand = new Random();
    int n = rand.nextInt(10);

    Scanner sc= new Scanner(System.in);
    int nb=-1;

    while(n!=nb){
        System.out.println("entrez un nombre (entre 1 et 10) :");
        nb = sc.nextInt();
        if(nb<n){
            System.out.println("Plus grand");
        }else
            if(nb>n) {
                System.out.println("Plus petit");
            }
    };
    System.out.println("Bravo !");
    sc.close();
    }
}
```